Factory Method

Factory Method Pattern

Let subclasses choose the type

DEFINITION

Define an interface for creating an object, but let subclasses decide which class to instantiate.

Factory Method puts a single "make the product" step behind a method the base class calls but subclasses override. The base class is written entirely against the Product interface and never names a concrete class; each subclass picks the concrete type by overriding that one method. Creation varies by subclassing, not by editing the caller.

WHY IT MATTERS

When a class hard-codes new ConcreteThing(), it is welded to that type — adding a variant means editing the class and risking its callers. Isolating creation in one overridable method lets new products arrive as new subclasses while the base algorithm stays closed for modification (OCP). The base depends only on the abstraction, so it is testable with a stub product.

How it works

Creator	Declares the factory method and calls it inside its own logic, coded against Product.
ConcreteCreator	Overrides the factory method to return one specific ConcreteProduct.
Product	The interface the Creator uses; ConcreteProduct is what actually gets built.

HOW TO APPLY

1. Find the new ConcreteX() that ties a class to one type.
2. Declare a factory method returning the Product interface; call it where you had new.
3. Move each concrete choice into a subclass that overrides the factory method.
4. Select the subclass once at the edge (config / DI); the base logic never names a concretion.

> LITMUS TEST
>
> Does this class call new on a type its subclasses might need to vary? If the algorithm is shared but the product differs, defer the new to an overridable method.

Vasyl Krupka · Ukraine

— Smell → fix

Dialog hard-codes a Windows button, so every new platform edits this class:

```
class Dialog {
render() {
const b = new WinButton() // welded to one type
b.paint()
}
}
```

↓ DEFER new TO A FACTORY METHOD

```
abstract class Dialog {
abstract createButton(): Button // factory method
render(){ this.createButton().paint() }
}
class WinDialog extends Dialog {
createButton(){ return new WinButton() }
}
```

Dialog.render() now depends only on Button; each platform is a subclass that overrides createButton(). A WebDialog adds a class — Dialog itself never changes.

USE WHEN

- ✓ A class can't anticipate the exact type it must create
- ✓ Localize object creation behind one overridable method

AVOID WHEN

- ▲ Only one product type will ever exist — just use new

Vasyl Krupka · Ukraine

— Trade-offs & pitfalls

PROS

+ Decouples client from concrete classes
+ OCP for new products

CONS

− A subclass per product
− More indirection

COMMON PITFALLS

▲ Reaching for Factory Method when a plain new or a simple static factory would do — a subclass per product is real overhead.

▲ Confusing it with Abstract Factory: Factory Method makes one product via inheritance; Abstract Factory makes families via composition.

▲ A parallel subclass hierarchy (one Creator per Product) that grows faster than the variation actually justifies.

WHERE YOU SEE IT

- A collection's iterator() returning its own Iterator type without the caller naming it.
- An abstract Dialog or Logger base whose subclasses pick the concrete Button or Transport.
- A framework "hook" method (createX) that apps override to plug in their implementation.

SEE ALSO

Abstract Factory	scales Factory Method up: a whole family of products instead of one
Template Method	the base algorithm that typically calls the factory method (GoF)
Prototype	an alternative to subclassing — clone a registered instance instead of overriding new
OCP	Factory Method is how you add product variants without editing the creator

Vasyl Krupka · Ukraine

Interview & sources

Factory Method vs Abstract Factory?

Factory Method makes one product by subclassing; Abstract Factory makes families of related products by composition — and is often built out of several factory methods.

Factory Method vs a simple static factory?

A static factory is just a named creator on a class; Factory Method is polymorphic — subclasses override which concrete type is returned.

How does it relate to Template Method?

The factory method is usually called by a template method: the base defines the algorithm skeleton and defers the "create" step to subclasses.

Isn't it over-engineering?

Often, for one product type — prefer plain new. It pays off when creation must vary by subtype and you want the base closed to edits.

REMEMBER

Define one overridable creation method; let subclasses decide which concrete product new returns.

SOURCES

GoF — "Design Patterns" (1994): Factory Method (creational)

Joshua Bloch — "Effective Java" (3rd ed., 2018): Item 1 — static factory methods (a related, distinct idiom)